Universite Paris Dauphine, PSL

Master 1 MIAGE

Solving the Rubik's Cube

with the A* Search Algorithm

Course: Artificial Intelligence and Reasoning

Lab 1, Project Report

Authors

Mathias CHEBBAH

Lamine BETRAOUI

Bouna SALL

June 2026

Abstract

This report presents a program that solves the Rubik's Cube. We model the cube as a search problem. We solve it with the A* algorithm of Lecture 2. We first run a uniform cost search. We then add two admissible heuristics. They guide the search. The first heuristic lets each sticker move on its own. The second one lets each piece move on its own. We measure how scrambled a cube can be. The results confirm the course. A good heuristic gives large savings. A* also runs out of memory before it runs out of time.

Contents

1 Introduction

A Rubik's Cube is a small cube. Each of its six faces holds nine colored stickers. Every face can turn, which mixes the colors. The cube is solved when each face shows a single color. Our goal is to write the "solve" button. The program takes any mixed cube. It must return a list of moves to the solved state.

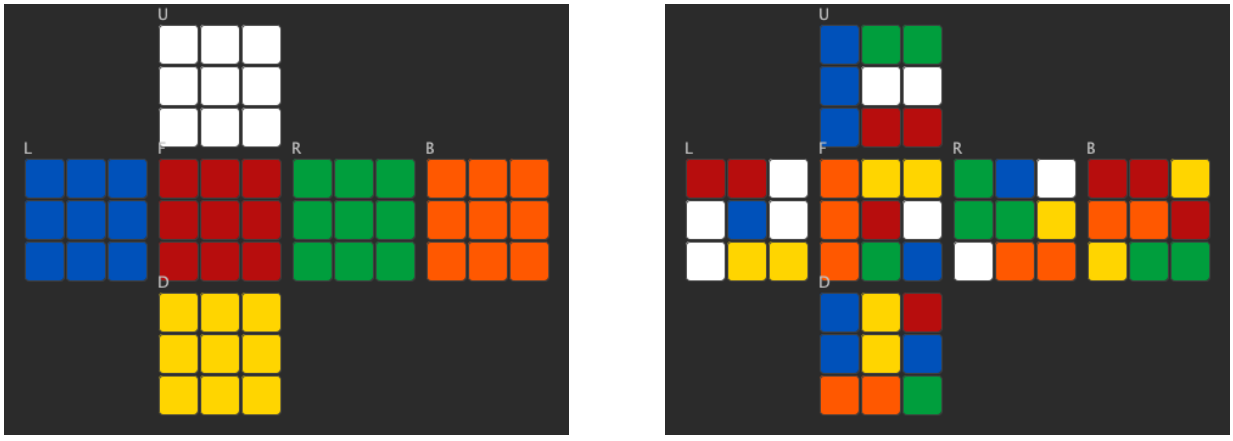
This problem fits the search framework of the course. A cube configuration is a state. A quarter turn of a face is an action. The solved cube is the goal. Every move costs one. So the shortest sequence of moves is the optimal solution. We therefore use the search methods of Lecture 2. We use A* in particular. It returns an optimal solution when its heuristic is admissible.

The report is organized as follows. Section 2 states the problem with the five components of a search problem. Section 3 describes the classes of the program. Section 4 explains A* and our implementation. Section 5 presents the two heuristics and shows that they are admissible. Section 6 reports the experiments. Section 7 concludes.

2 Problem formulation

2.1 The cube and its representation

We follow the Western color scheme. In the solved cube, UP is white and DOWN is yellow. FRONT is red and BACK is orange. LEFT is blue and RIGHT is green. Opposite faces are UP and DOWN, FRONT and BACK, LEFT and RIGHT. Each face is a three by three matrix of characters. The characters are 'W', 'R', 'G', 'B', 'Y' and 'O'. Figure 1 shows a solved cube and a scrambled cube. Our interface draws them as an unfolded net.



(a) Solved cube. (b) Cube after a few random moves.

Figure 1: The cube shown by our interface as an unfolded net. The center of each face never moves, so it keeps its solved color.

Inside the program the 54 stickers are stored in one flat array. The matrix of a face is a view over this array. Each face uses row and column indices from 0 to 2. This follows the figure of the statement. To turn a face, we must know where every sticker goes. We place each sticker in a 3D frame (Figure 2). The cube is centered at the origin. A turn is then a real rotation of one layer. This way the permutation is computed, not typed by hand. So it cannot contain a wrong strip.

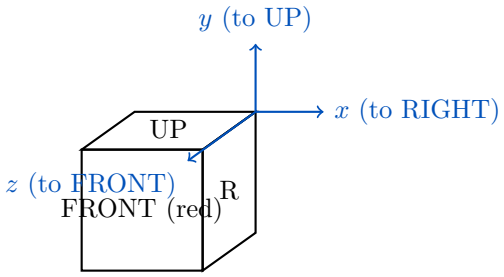


Figure 2: The 3D frame used to compute the turns. The outward normal of each face points along one axis. A clockwise turn rotates one layer by 90 degrees.

2.2 The five components

We define the search problem with the five components of Lecture 2.

- **Initial state.** The scrambled cube given by the user.
- **Actions.** The twelve quarter turns. Six are clockwise (U, L, F, R, D, B). Six are counter clockwise (U', L', F', R', D', B').
- **Transition model.** Applying a turn produces a new cube. The result of an action permutes the stickers of one face and its belt.
- **Goal test.** The cube is solved when every face is uniform with its solved color.
- **Path cost.** Each move costs one. The cost of a path is its number of moves.

The state space is huge. A real cube has about 4.3×10^{19} reachable configurations. We cannot store them all. A* must therefore explore as few states as possible. This is exactly why a good heuristic matters.

2.3 The twelve actions

Each action has an integer identifier, from 0 to 11, as in the statement. Table 1 lists them with their notation.

Id	Method	Notation	Sense	Id	Method	Notation	Sense
0	turnUp	U	clockwise	6	returnUp	U'	counter
1	turnLeft	L	clockwise	7	returnLeft	L'	counter
2	turnFront	F	clockwise	8	returnFront	F'	counter
3	turnRight	R	clockwise	9	returnRight	R'	counter
4	turnDown	D	clockwise	10	returnDown	D'	counter
5	turnBack	B	clockwise	11	returnBack	B'	counter

Table 1: The twelve actions and their identifiers.

2.4 Why this search is hard

The course measures a search by three quantities. The branching factor b is the number of children of a node. The depth d is the depth of the shallowest goal. The maximum path length is m. For our problem b is twelve. Each state has twelve children. The number of nodes in a blind search grows like b^d. With $b = 12$, even a small depth gives a huge tree. For example, 12^7 is more than thirty million. So a uniform cost search cannot go deep. We need a heuristic.

Measure	Symbol	Value for our problem
Branching factor	b	12 (twelve turns)
Worst case time	$O(b^d)$	exponential in the depth
Worst case space	$O(b^d)$	the real limit we hit

Table 2: Complexity terms of the course applied to the cube.

3 Program architecture

The code is written in Java. We chose Java for several reasons. The statement describes a graphical interface and an interface for heuristics. It also uses Java collections, such as the linked list. The classes follow the statement closely. They are grouped by role, as Table 3 shows.

Package	Class	Role
model	`Cube`	Six face matrices, the twelve turns, isSolved
model	`Face`, `Color`	Faces, colors and their solved values
model	`Move`	Notation and inverse of each action
search	`State`	A node of the search tree, with expand
search	`Astar`	The A* algorithm and the solve method
search	`StateSet`	The explored set, with add and contains
search	`PriorityQueueState`	The frontier, with push and pop
heuristic	`Heuristic`	The heuristic interface, with value
heuristic	`HeuristicLabel`	The sticker relaxation heuristic
heuristic	`HeuristicCube`	The piece relaxation heuristic
ui	`RubiksCubeSimulator`	The interface and the solve button

Table 3: Main classes of the project, grouped by package.

3.1 The Cube class

The `Cube` class models the game. It stores the 54 stickers. It offers the twelve turns and the `isSolved` test. The hard part is to know where each sticker goes. Writing these tables by hand is error prone. Instead, we give each sticker a 3D position and a face normal. We then rotate the affected layer by 90 degrees. We read the permutation back from the rotated positions. This is correct by construction.

We checked the model with algebraic invariants. A turn followed by its inverse is the identity. The same turn four times is the identity. A turn equals three turns the other way. The number of stickers of each color stays nine. The sequence R U R' U' has order six. It returns to the solved cube after six repetitions. All these tests pass.

3.2 The State class

A `State` is a node of the search tree. It stores the cube and the cost g from the root. It also stores the parent and the action used to reach it. Finally it stores the heuristic value h. The constructor computes h once. The `expand` method applies the twelve turns. It returns the twelve children in a linked list, as the statement asks. The Java code below shows the method.

```
public LinkedList<State> expand() {
    LinkedList<State> children = new LinkedList<>();
    for (int action = 0; action < 12; action++) {
        Cube next = cube.withAction(action);
        children.add(new State(next, nbrActions + 1, this, action));
    }
    return children;
}
```

Each child keeps a reference to its parent and to its action. These two fields are called `pere` and `actionPere` in the statement. They let us rebuild the solution once the goal is reached. We compare two states by their cube only. Two paths to the same configuration are therefore the same state. This lets the explored set and the frontier remove duplicates.

3.3 The explored set and the frontier

The `StateSet` class is the explored set. It offers `add` and `contains`. It also keeps the smallest cost seen for each configuration. This lets A* skip a configuration already reached by a cheaper path. The `PriorityQueueState` class is the frontier. It offers `push` and `pop`. The `pop` method returns the state with smallest $f = g + h$. The `push` method checks if the configuration is already there. It then keeps only the better one.

3.4 The graphical interface

The `RubiksCubeSimulator` class is the interface. It holds a cube and draws it as a net (Figure 1). It has a `main` method that creates the window. Its `actionPerformed` method reacts to every button. There is one button per turn, plus scramble, reset and solve. A small menu picks the strategy.

When the user presses solve, A* runs on a copy of the cube. The search runs on a background thread, so the window stays responsive. The interface then plays the plan move by move, like a short animation. The log panel shows the plan, the expanded states, and the time. During the search, the move buttons are disabled. So the user cannot change the cube under the solver. Figure 3 shows the full window.

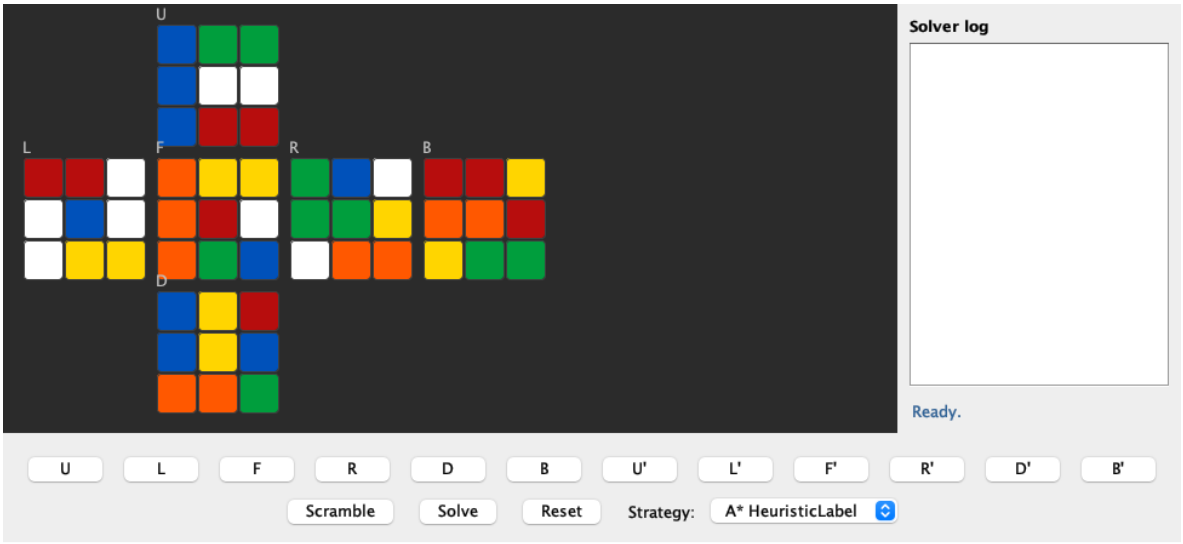


Figure 3: The simulator window. The cube net is on the left, the solver log on the right, the twelve move buttons and the solve button below, with a menu to pick the strategy.

4 The A* algorithm

4.1 Principle

A* is an informed search. It expands the node with the smallest evaluation function

$$f(n) = g(n) + h(n),$$

Here $g(n)$ is the cost from the start to n. And $h(n)$ estimates the cost from n to the goal. An admissible h never overestimates the real cost. With such an h, A* returns an optimal solution. With $h = 0$ the function f becomes $f = g$. So A* behaves as a uniform cost search.

The course presents several search strategies. Table 4 compares the ones we use. The uniform cost search is optimal but blind. It explores by cost only. A* keeps the same guarantees and adds the heuristic. So it looks toward the goal. This is why A* expands far fewer states.

Strategy	Orders the frontier by	Optimal	Uses a heuristic
Breadth first search	depth	yes (unit cost)	no
Uniform cost search	g	yes	no
Greedy best first	h	no	yes
A*	$f = g + h$	yes (admissible h)	yes

Table 4: Search strategies of the course. We use uniform cost search and A*.

Figure 4 shows a small search tree. Each node carries its values g, h and f. A* always expands the leaf with the smallest f. A node that looks cheap by g alone may still be skipped. Its heuristic may be large.

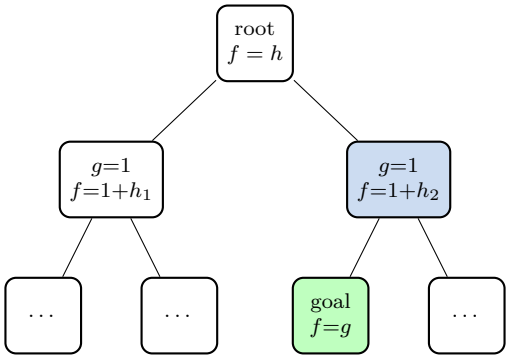


Figure 4: A small A* search tree. The frontier is ordered by $f = g + h$. The shaded node has the smallest f, so it is expanded first and reaches the goal.

4.2 The solve method

The `solve` method follows the A* pseudo code of Lecture 2. It uses the frontier and the explored set. It tests the goal when a node is popped. This keeps the search optimal. It then rebuilds the plan by walking the parent chain. Algorithm 1 shows the method.

Algorithm 1 The A* search, as implemented in solve.

```
1:  frontier.push(root)
2:  while frontier is not empty do
3:      node ← frontier.pop()                          ▷ smallest f = g + h
4:      if node already expanded with a smaller g then
5:          continue
6:      end if
7:      explored.add(node)
8:      if node is the goal then
9:          return the plan (walk the parent chain)
10:     end if
11:     for all child in node.expand() do              ▷ the 12 turns
12:         if child already expanded with a smaller g then
13:             continue
14:         end if
15:         frontier.push(child)
16:     end for
17: end while
18: return failure
```

4.3 Reconstructing the solution

When the goal is found, we follow the parent links to the root. At each step we read the action that produced the node. We reverse the order. This gives the moves from the start to the solved cube. The `solve` method returns them as strings, such as "R" or "U'".

4.4 Why A* is optimal

The course states that A* is optimal with an admissible heuristic. The idea is simple. A* tests the goal only on the node with smallest f. Suppose this goal node has cost g. The goal is reached, so h is zero there. Thus $f = g$. Any other node n in the frontier has $f(n) = g(n) + h(n)$. Since

h never overestimates, $f(n)$ is a lower bound. It bounds any solution through n. As n was not chosen, $f(n) \geq g$. So no cheaper solution can exist. The first goal popped is therefore optimal. We also keep an explored set. A configuration reached again by a longer path is skipped. This is the graph version of A*.

4.5 Data structures

The frontier is a priority queue ordered by f. Java does not offer a cheap way to lower a key already in the queue. So when a better path appears, we push a new entry. We mark the old one as out of date. Out of date entries are skipped when popped. The explored set is a hash map. It maps a configuration to its best known cost. Both structures compare states by their cube. So they detect repeats and keep the search from looping.

5 Heuristics

A heuristic must be admissible to keep A* optimal. Both of our heuristics come from a relaxation of the problem. We saw this idea in the tutorial on informed search. A relaxation removes some rules. Its optimal cost is then a lower bound on the real cost.

5.1 HeuristicCube: the piece relaxation

This heuristic assumes that each piece moves on its own. A piece is a corner or an edge. A corner has three stickers and an edge has two. The six centers never move. A piece is well placed when all its stickers are on target. A single turn moves four corners and four edges. So one turn fixes at most four corners. It also fixes at most four edges. The number of remaining moves is therefore at least

$$h_{\text{cube}} = \max\left(\left\lceil \frac{\text{misplaced corners}}{4} \right\rceil, \left\lceil \frac{\text{misplaced edges}}{4} \right\rceil\right).$$

The maximum of two lower bounds is still a lower bound, so this heuristic is admissible.

5.2 HeuristicLabel: the sticker relaxation

This heuristic assumes that each sticker moves on its own. A sticker only needs to reach its correct face. We measure the 3D Manhattan distance to its target face. We count it in quarter turns. The distance is 0 on the correct face. It is 1 on an adjacent face and 2 on the opposite face. We sum this distance over all 54 stickers. We then divide by the stickers moved per turn, which is twelve.

$$h_{\text{label}} = \frac{1}{12} \sum_{s=1}^{54} d(s).$$

One turn moves twelve belt stickers. Each one moves to an adjacent face. So each one cuts its distance by at most one. The total distance drops by at most twelve per move. The remaining moves are therefore at least the sum divided by twelve. So this heuristic is admissible.

5.3 Checking admissibility

We checked both heuristics by tests. Take a cube scrambled with k random moves. The heuristic must not exceed k. Undoing the scramble takes at most k moves. We ran this check for many seeds and depths. The value was always at most k. Both heuristics also return zero on the solved

cube. A stronger check appears in the experiments. A* with these heuristics matches the uniform cost search length. This proves that they keep the search optimal.

5.4 A small worked example

Take a solved cube and apply one move, the turn F. This move sends twelve belt stickers to an adjacent face. For the sticker heuristic, each one is now next to its target. So its distance is one. The sum is twelve, and $h_{\text{label}} = 12/12 = 1$. The true cost is also one. So the estimate is exact here. For the piece heuristic, this move misplaces four corners and four edges. So the value is $\max(\lceil 4/4 \rceil, \lceil 4/4 \rceil) = 1$. Both heuristics agree with the real cost of one move.

5.5 Comparing the two heuristics

A heuristic h_a dominates h_b when $h_a(n) \geq h_b(n)$ for every state n. Both must stay admissible. A dominating heuristic is never worse. It usually expands fewer states. In our tests the sticker heuristic gives larger values. So it guides the search better. The experiments in Section 6 confirm this. The sticker heuristic expands fewer states at the same depth. Both stay admissible, so both keep A* optimal.

6 Experiments

6.1 Setup

We measured three strategies. The first is the uniform cost search, where $h = 0$. The second is A* with the piece heuristic. The third is A* with the sticker heuristic. For each strategy and depth, we solved several cubes. We recorded the success rate and the average expanded states. We also recorded the average time and solution length. A run fails when it passes a fixed node limit. The seeds are fixed, so the numbers can be reproduced.

6.2 Results

Table 5 reports the main numbers. The pattern is clear. The uniform cost search blows up around depth seven. It expands hundreds of thousands of states at depth six. It then fails at depth seven and eight under the limit. Both heuristics scale much further. The sticker heuristic is the strongest here. It expands far fewer states than the piece heuristic.

	UCS ($h = 0$)		A* HeuristicCube		A* HeuristicLabel	
Depth	expanded	time (ms)	expanded	time (ms)	expanded	time (ms)
4	6 818	26	8	0.1	14	0.2
5	20 599	86	64	0.7	40	0.9
6	503 575	3 170	450	5.2	400	5.7
7	fails	n/a	6 953	66	1 597	16
8	fails	n/a	26 304	207	5 793	56

Table 5: Average expanded states and time per strategy and scramble depth. UCS fails from depth seven under the node limit.

Figure 5 draws the expanded states on a logarithmic scale. The gap between the uniform cost search and A* grows very fast. At depth six the gap is about three orders of magnitude. The two A* curves stay low and close. The sticker heuristic is slightly below the piece heuristic.

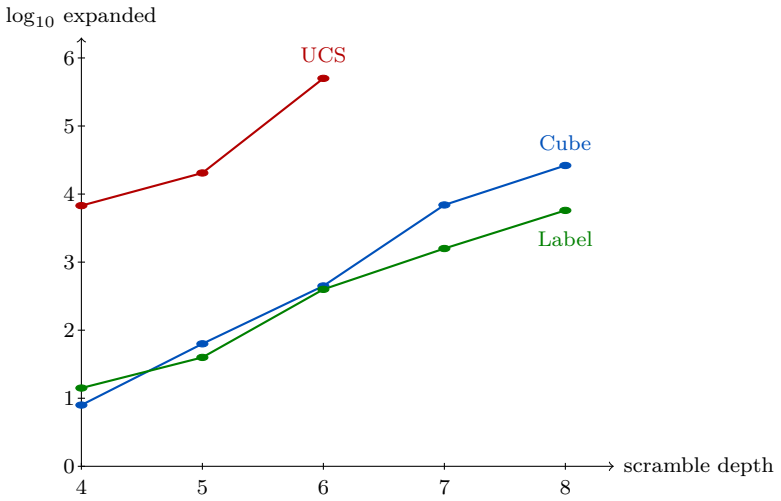


Figure 5: Expanded states by depth, log scale. UCS grows much faster and stops at depth six within memory. Both A* curves stay low.

6.3 A complete example

We scramble a solved cube with seven moves. We solve it with the sticker heuristic. Table 6 shows the scramble, the plan, and the effort. The plan has seven moves, so it is optimal. The plan is also the scramble read backward. Each move is replaced by its inverse. This is the shortest way to undo the scramble. A* finds it.

Scramble (7 moves)	`F U' L U R' D L'`
Solution (7 moves)	`L D' R U' L' U F'`
Expanded states	1 110
Generated states	12 030
Time	about 20 ms

Table 6: A complete solve with the sticker heuristic. The plan undoes the scramble in the optimal number of moves.

6.4 Optimality

The solution length is the same for all three strategies at every depth. For example, the average length is 6.0 at depth six. It is 7.6 at depth eight, for all three. This is an empirical proof that the heuristics are admissible. An overestimating heuristic would make A* return a longer plan. This never happens.

6.5 Discussion

The experiments confirm the course. First, the heuristic does not change the answer. It only changes the effort. The plans stay optimal. Second, a good heuristic gives huge savings. At depth six, the uniform cost search expands about 500 000 states. A* expands a few hundred. Third, the sticker heuristic dominates the piece heuristic here. It gives finer values. Finally, A* runs out of memory before it runs out of time. This matches the slides. The first limit is the stored states, not the clock.

6.6 How far A* can go

We pushed the sticker heuristic, our strongest one, to deeper scrambles. Table 7 shows the cost as the depth grows. The effort grows fast, about ten times per added move. Depth eight is solved in a tenth of a second. Depth ten still works. But it expands half a million states. It then takes about twelve seconds. Beyond depth ten the frontier no longer fits in memory. So our solver stays practical up to about nine or ten random moves. This is the answer to the question of the statement.

Scramble depth	Expanded states	Solution length	Time (s)
8	6 818	7.0	0.1
9	71 359	8.7	1.7
10	495 949	9.3	11.8

Table 7: A* with the sticker heuristic on deeper scrambles, averaged over six cubes per depth. The cost grows by about an order of magnitude per move.

6.7 A simple optimization

We also added one optional optimization. After a move, we never apply its inverse right away. For example we never play U' just after U. Such a pair cancels out, so it can never belong to an optimal solution. Skipping it keeps the search optimal and makes the tree smaller. This option is off by default. So the `solve` method stays close to the course pseudo code. When it is on, A* expands fewer states at the same depth.

6.8 Methodology

We built the project with a test first method. For each class we wrote the tests before the code. We made the tests fail, then wrote the code until they passed. The cube tests check the algebraic invariants. The search tests check that the plan really solves the cube. They also check that A* matches the optimal length. In total the project has twenty four tests, and they all pass. The code was also reviewed. The review helped us find and fix a rotation that turned the wrong way. It also pointed out a few performance issues.

6.9 Limits

A* is still exponential in the worst case. Our solver stays fast up to about eight or nine random moves. Beyond that, the frontier grows too large for memory. This is a known limit of A* on the Rubik's Cube. Stronger methods exist, but they are outside the scope of this lab.

7 Conclusion

We built a program that solves the Rubik's Cube with the A* algorithm. We modeled the cube as a search problem with the five components of the course. We followed the statement and built every required class. We first ran a uniform cost search. We then added two admissible heuristics. Both come from a relaxation of the problem.

The results match the theory. A* returns an optimal solution. A good heuristic cuts the number of expanded states by several orders of magnitude. The uniform cost search fails from about seven random moves. A* solves more scrambled cubes. The main limit is memory, not time. This project gave us a clear view of informed search. It showed us why heuristics matter.

Team contributions

The three members worked together on the design and the report. The work was shared as follows. One member focused on the cube model and the turns. One member focused on the A* search and the data structures. One member focused on the two heuristics and the experiments. The interface, the tests, and the writing were a shared effort. The members reviewed each other's work regularly.